

Esame OS Internals (Cabodi)

Data: 15/02/2021

QUESTIONS

SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale, TLB e EAT)

Contesto: Sia dato un sistema di memoria virtuale con paginazione, nel quale vengono indirizzati i Byte. Il sistema dispone di TLB (Translation Look-aside Buffer). La tabella delle pagine ("page-table") viene realizzata con uno schema a due livelli, nel quale un indirizzo logico di 64 bit viene suddiviso (da MSB a LSB) in 3 parti: p_1 , p_2 e d , rispettivamente di 40 bit, 12 bit e 12 bit. Non si utilizzano ulteriori strutture dati (quali tabelle di hash o inverted page table) per velocizzare gli accessi. La memoria virtuale viene gestita con paginazione a richiesta.

Si risponda alle seguenti domande:

-) Supponendo che la memoria RAM abbia tempo di accesso di 160 ns, si calcoli il TLB miss ratio (frequenza di miss), tale da garantire effective access time $EAT_{PT} \leq 200$ ns, assumendo che il tempo di accesso alla TLB sia trascurabile. Si tratta di un lower bound o di un upper bound per la TLB miss ratio?
 -) Si consideri ora la frequenza di page fault $p = 10^{-5}$. In quanto tempo T viene servito un page fault se si osserva che il degrado nel tempo di accesso dovuto ai soli PF è superiore al 10%? Si tratta di un valore massimo o minimo per T ? (Siccome H non è noto, ma è dato da un intervallo di valori, si calcoli T per H minimo e massimo).
 -) Data una stringa di riferimenti a indirizzi in memoria (letture e scritture), il numero di page fault può variare al variare della dimensione di una pagina? (Motivare). A parità di tutto il resto, può servire modificare la dimensione (numero di bit) di p_1 e p_2 (senza cambiare d) per ridurre il numero di PF? (Motivare).
-

Domanda 2 (File System, Inodes e Blocchi)

Contesto: Considerare un file system Unix-like, basato su inode, con 13 puntatori / indici (10 diretti, 1 singolo indiretto, 1 doppio indiretto e 1 triplo indiretto). I puntatori / indici hanno una dimensione di 32 bit e i blocchi del disco hanno una dimensione di X Byte. Il file system risiede su una partizione del disco di 800 GB, che include sia blocchi di dato che blocchi di indice.

Si risponda alle seguenti domande:

-) Supponendo che tutti i metadati (eccetto i blocchi indice) abbiano dimensioni trascurabili, calcolare i possibili valori di X , tenendo conto che si tratta di una potenza di 2 e che il file system deve garantire di poter contenere fino a 100 file con indicizzazione indiretta tripla. Si calcolino i valori minimo e massimo per X .
-) Dato un file binario di dimensione 10381.25 KB, calcolare esattamente quanti blocchi indice e blocchi di dato occupa il file, corrispondenti ai valori minimo e massimo precedentemente calcolati per X .

Domanda 3 (Dynamic Loading, IPT e TLB)

Contesto: Si risponda alle seguenti domande sulla gestione della memoria:

-) Si consideri il caricamento dinamico (dynamic loading) e il link dinamico (dynamic linking). È possibile caricare dinamicamente un programma senza che sia necessario il dynamic linking? Il dynamic linking richiede che un programma sia anche caricabile dinamicamente (dynamic loading)?
-) Si spieghi brevemente perché un’Inverted Page Table (IPT) necessiti di una tabella di HASH. Perché la soluzione di IPT + tabella di HASH è diversa da una soluzione con PT basata solamente su tabella di Hash?
-) Si consideri una CPU dotata di TLB: la TLB può contenere entry di più processi simultaneamente o è vincolata a contenere entry per un solo processo?

SECTION 2: OS161

Domanda 4 (Configurazione e Compilazione Kernel)

Contesto: Sia dato un sistema OS161. Si supponga di aver aggiunto a `kern/conf/conf.kern` le istruzioni:

```
defoption exam
optfile exam syscall/exam.c
```

e di aver creato in `kern/conf` il file `EXAM`, copiato dal file `DUMBVM`.

Si risponda alle seguenti domande:

-) Si dica se sono sufficienti le azioni appena descritte, più l'esecuzione in `kern/conf` di `./config EXAM`, affinché venga compilato il file opzionale `syscall/exam.c` quando si effettuano, in `kern/compile/EXAM`, i comandi `bmake depend` e `bmake`.
-) Quale, tra i file `exam.h` e `opt-exam.h`, viene generato automaticamente dal comando `./config EXAM`? Viene generato sempre, oppure solo se in `EXAM` compare l'istruzione `options exam`?
-) Cosa contiene il file (quello generato automaticamente, citato nella domanda precedente)?
-) Si supponga di inserire, nel file `main.c`, l'istruzione `exam_init();`. Tenendo conto che è realizzata nel file `syscall/exam.c`, come è possibile fare in modo che l'istruzione sia presa in considerazione e compilata solamente nelle versioni del kernel in cui viene abilitata l'opzione `exam`, mediante le istruzioni seguenti?

```
#ifdef OPT_EXAM
    exam_init();
#endif
```

Dire anche cosa è necessario includere in un file C per poter usare questo tipo di opzioni.

Domanda 5 (Processi, Trapframe e Memoria Mappata)

Contesto: Si consideri un processo utente su OS161.

Si risponda alle seguenti domande:

-) Spiegare brevemente la differenza tra `switchframe` e `trapframe`. Quale viene utilizzato per avviare un processo utente? Quale per gestire una chiamata di sistema? Quale per la `thread_fork`?
-) Perché i dispositivi di I/O sono mappati su `kseg1`? Sarebbe possibile mappare un dispositivo di I/O in `kseg0`?
-) Un processo utente può eseguire una `write(fd, buf, nb)`, dove `buf` è un indirizzo in `kseg0`?

ANSWERS

Domanda 1 (Memoria Virtuale, TLB e EAT)

A) Calcolo del TLB Miss Ratio:

In un'architettura con Page Table a 2 livelli, un TLB Miss richiede 2 accessi extra in memoria per la traduzione, seguiti dall'accesso al dato.

- Tempo TLB Hit = $T_{RAM} = 160$ ns
- Tempo TLB Miss = $2 \cdot T_{RAM} + T_{RAM} = 3 \cdot T_{RAM} = 480$ ns

Sia M il Miss Ratio. L'Effective Access Time base è dato da:

$$EAT_{base} = (1 - M) \cdot 160 + M \cdot 480 = 160 + 320M$$

Imponendo il vincolo $EAT \leq 200$ ns:

$$160 + 320M \leq 200 \implies 320M \leq 40 \implies M \leq \frac{40}{320} = \mathbf{0.125} \text{ (12,5\%)}$$

Poiché l'equazione definisce un limite superiore ($\leq$) affinché l'EAT resti entro la soglia desiderata, si tratta di un **Upper Bound (valore massimo)** per la TLB miss ratio.

B) Calcolo del tempo di servizio del Page Fault (T):

La formula completa che include il costo dei Page Fault è: $EAT_{PF} = (1 - p)EAT_{base} + p \cdot T$. La traccia richiede che il degrado sia superiore al 10% di EAT_{base} :

$$EAT_{PF} - EAT_{base} > 0.1 \cdot EAT_{base} \implies p(T - EAT_{base}) > 0.1 \cdot EAT_{base}$$

Isolando T :

$$T > EAT_{base} \cdot \left(\frac{0.1 + p}{p} \right)$$

Sapendo che $p = 10^{-5}$, calcoliamo i due estremi derivanti dall'intervallo di Miss Ratio:

1. **Caso M massimo** ($M = 0.125 \implies EAT_{base} = 200$ ns):

$$T > 200 \cdot \left(\frac{0.10001}{0.00001} \right) = 200 \cdot 10001 = \mathbf{2.000.200} \text{ ns}$$

2. **Caso M minimo** ($M = 0 \implies EAT_{base} = 160$ ns):

$$T > 160 \cdot \left(\frac{0.10001}{0.00001} \right) = 160 \cdot 10001 = \mathbf{1.600.160} \text{ ns}$$

Poiché le disequazioni impongono che T debba essere *maggiore* di tali soglie affinché si verifichi il degrado del 10%, i risultati ottenuti rappresentano un **valore minimo (Lower Bound)** per T .

C) Effetti della variazione di pagina e struttura PT:

- **Modifica della dimensione della pagina: SÌ, influisce sul numero di PF.** Tuttavia l'effetto non è lineare. Aumentare la dimensione riduce inizialmente i Page Fault grazie al principio di località spaziale. Se però la pagina diventa eccessivamente grande, diminuisce il numero di frame fisici disponibili per il processo e aumenta la frammentazione interna, portando inevitabilmente a un *aumento* vertiginoso dei Page Fault (fenomeno del *thrashing*).
- **Modifica di p1 e p2 (senza cambiare d): NO, non influisce minimamente.** La dimensione d rappresenta l'offset, che definisce in modo univoco la dimensione della pagina logica e fisica (2^d). I parametri $p1$ e $p2$ determinano esclusivamente come l'indirizzo viene sezionato per navigare la struttura dati della Page Table nella memoria. Alterarli modificherà l'ingombro in RAM della Page Table stessa, ma la mappatura degli indirizzi all'interno delle pagine logiche resterà inalterata, generando l'identica sequenza di Page Fault.

Domanda 2 (File System Unix-like, Inodes e Blocchi)

A) Valori minimo e massimo per la dimensione del blocco X:

La dimensione X deve essere una potenza del 2. Esistono due vincoli architetturali che definiscono il range di X .

1. Calcolo del limite inferiore (X_{min}): Vincolo di indirizzabilità

La partizione ha dimensione 800 GB $\approx 800 \times 2^{30}$ Byte. I puntatori sono di 32 bit, ovvero possono indirizzare un massimo di 2^{32} blocchi logici. Il numero totale di blocchi nel disco deve essere minore o uguale al massimo indirizzabile:

$$\frac{800 \times 2^{30}}{X} \leq 2^{32} \implies X \geq \frac{800 \times 2^{30}}{2^{32}} = \frac{800}{4} = 200 \text{ Byte}$$

Essendo X una potenza di 2, il valore minimo valido è $X_{min} = 256 \text{ Byte}$.

2. Calcolo del limite superiore (X_{max}): Vincolo di spazio per 100 file in tripla indirezione

La partizione deve poter ospitare 100 file che iniziano a usare l'indicizzazione tripla. Un file innesca il livello triplo quando esaurisce i puntatori diretti (10), singoli ($X/4$) e doppi ($(X/4)^2$). Il numero di blocchi dati minimi per raggiungere l'indirezione tripla è:

$$\text{Blocchi Dato}_{th} = 10 + \frac{X}{4} + \left(\frac{X}{4}\right)^2 + 1$$

Procediamo per tentativi testando le classiche potenze di 2, sapendo che lo spazio per 100 file deve essere $\leq 800 \text{ GB}$:

- **Prova con $X = 4096 \text{ B (4 KB)}$:** Puntatori per blocco = 1024.
 Blocchi Dato = $10 + 1024 + 1024^2 + 1 \approx 1.049.611$ blocchi.
 Spazio totale per 1 file $\approx 1.049.611 \times 4096 \text{ B} \approx 4,3 \text{ GB}$.
 Spazio per 100 file $\approx 430 \text{ GB}$. Poiché $430 \text{ GB} \leq 800 \text{ GB}$, la condizione è **Soddisfatta**.
- **Prova con $X = 8192 \text{ B (8 KB)}$:** Puntatori per blocco = 2048.
 Blocchi Dato = $10 + 2048 + 2048^2 + 1 \approx 4.196.363$ blocchi.
 Spazio totale per 1 file $\approx 4.196.363 \times 8192 \text{ B} \approx 34,3 \text{ GB}$.
 Spazio per 100 file $\approx 3430 \text{ GB}$. Questo **supera ampiamente gli 800 GB** disponibili.

Pertanto, il valore massimo valido è $X_{max} = 4096 \text{ Byte (4 KB)}$.

B) Calcolo dei blocchi dato e indice per file da 10381,25 KB:

Dimensione file in byte = $10381,25 \times 1024 = 10.630.400 \text{ Byte}$.

Caso 1: Con $X_{max} = 4096 \text{ Byte (1024 puntatori/blocco)}$

- **Blocchi dato:** $\lceil 10.630.400/4096 \rceil = \mathbf{2596}$ blocchi dato.
- **Blocchi indice:** I 2596 blocchi sono così mappati:
 - 10 tramite indici diretti (restano 2586 blocchi).
 - 1024 coperti da **1** blocco singolo indiretto (restano 1562 blocchi).
 - I 1562 restanti cadono nel doppio indiretto. Servono $\lceil 1562/1024 \rceil = \mathbf{2}$ blocchi interni di secondo livello, puntati da **1** blocco doppio esterno.

Totale blocchi indice = 1(singolo) + 1(doppio ext) + 2(doppi int) = **4** blocchi indice.

Caso 2: Con $X_{min} = 256$ Byte (64 puntatori/blocco)

- **Blocchi dato:** $\lceil 10.630.400/256 \rceil = \mathbf{41.525}$ blocchi dato.
- **Blocchi indice:** I 41525 blocchi sono così mappati:
 - 10 tramite indici diretti.
 - 64 tramite **1** blocco singolo indiretto.
 - $64 \times 64 = 4096$ coperti interamente dal doppio indiretto, che richiede: **1** blocco esterno e **64** blocchi interni (Totale 65).
 - I restanti $41.525 - (10 + 64 + 4096) = 37.355$ cadono nel triplo indiretto.
 - Per 37.355 blocchi in tripla indirezione servono: $\lceil 37.355/64 \rceil = \mathbf{584}$ blocchi singoli interni. A loro volta, questi 584 blocchi richiedono $\lceil 584/64 \rceil = \mathbf{10}$ blocchi doppi interni. A cui si somma **1** blocco triplo esterno.

Totale blocchi indice = 1(singolo)+65(doppio)+1(triplo ext)+10(doppi int)+584(singoli int) = **661** blocchi indice.

Domanda 3 (Dynamic Loading, IPT e TLB)

A) Dynamic Loading e Dynamic Linking:

- **È possibile caricare dinamicamente senza link dinamico? SÌ.**
 Il *Dynamic Loading* (caricamento dinamico) è una tecnica in cui una routine o un modulo viene caricato nella memoria principale (RAM) solo nel momento in cui viene esplicitamente chiamato durante l'esecuzione. Questa tecnica non richiede particolari supporti dal sistema operativo, ma è responsabilità del programmatore. Un programma può essere interamente collegato staticamente (*Static Linking*) al momento della compilazione, risultando in un file eseguibile molto grande su disco, ma le cui singole parti verranno caricate in memoria dinamicamente solo se e quando il flusso di esecuzione le attraverserà.
- **Il dynamic linking richiede il dynamic loading? SÌ.**
 Il *Dynamic Linking* rimanda la risoluzione degli indirizzi delle librerie al momento dell'esecuzione, utilizzando frammenti di codice chiamati *stub*. Quando lo stub viene eseguito per la prima volta, verifica se la libreria richiesta è già presente in memoria; in caso negativo, innesca il sistema operativo per caricarla. Questo meccanismo implica necessariamente un'operazione di *Dynamic Loading* della libreria stessa all'interno della RAM.

B) Inverted Page Table e Tabella di Hash:

- **Necessità della tabella di Hash:**

L'Inverted Page Table (IPT) è indicizzata in base ai frame fisici della memoria RAM, non in base alle pagine logiche. Quando la MMU deve tradurre un indirizzo virtuale, possiede la coppia chiave (PID, Logical Page). Per trovare in quale frame risiede tale pagina, la MMU dovrebbe eseguire una ricerca lineare (*scan*) dell'intera IPT, un'operazione con complessità $O(N)$ assolutamente proibitiva in termini di performance. L'introduzione di una tabella di Hash che utilizza (PID, Logical Page) come chiave permette di individuare il frame fisico corrispondente (o la catena di collisioni) in tempo $O(1)$, rendendo l'accesso compatibile con i requisiti hardware.

- **Differenza rispetto a una Hashed PT standard:**

In una Hashed Page Table tradizionale, la tabella di Hash rappresenta la struttura dati primaria di traduzione e la sua dimensione scala logicamente in base allo spazio di indirizzamento virtuale richiesto dai processi. Nella soluzione con IPT + Hash, la struttura primaria di riferimento è la IPT, la cui dimensione è globalmente fissa e strettamente limitata alla quantità di memoria fisica (RAM) installata nel sistema. La tabella di Hash funge unicamente da indice acceleratore per la struttura sottostante.

C) Capacità della TLB e processi simultanei:

La capacità della TLB di contenere entry per più processi dipende dall'architettura hardware della CPU.

- **Con ASID (Sì, più processi):** Se la CPU implementa un campo **ASID** (*Address Space Identifier*) all'interno delle singole entry della TLB, la memoria cache può contenere simultaneamente le traduzioni per più processi. La MMU confronterà l'ASID della entry con l'ASID del processo attualmente in esecuzione (es. salvato in un registro dedicato), ignorando i matching dei processi inattivi. Questo evita onerosi svuotamenti della TLB al cambio di contesto.
- **Senza ASID (No, un solo processo):** Se l'architettura non prevede l'ASID, le traduzioni salvate nella TLB sono indistinguibili. In questo scenario, la TLB può contenere unicamente le entry del processo in esecuzione e il Sistema Operativo è obbligato a eseguire un *TLB Flush* (svuotamento totale) a ogni *Context Switch* per garantire la sicurezza e l'isolamento degli spazi di indirizzamento.

Domanda 4 (Configurazione e Compilazione Kernel)

A) Sufficienza delle azioni per la compilazione del file opzionale:

NO, le azioni descritte non sono sufficienti.

Motivazione: Le istruzioni `defoption` e `optfile` aggiunte a `conf.kern` si limitano a dichiarare l'esistenza di una nuova opzione (`exam`) e ad associare ad essa il file sorgente da compilare (`syscall/exam.c`). Tuttavia, di default, tutte le opzioni sono disabilitate. Affinché il comando `bmake` decida effettivamente di compilare il file, è necessario abilitare esplicitamente l'opzione inserendo la direttiva `options exam` all'interno del file di configurazione specifico del kernel, ovvero il file `kern/conf/EXAM` (creato copiando `DUMBVM`). Solo dopo questa modifica il lancio di `./config EXAM` genererà i Makefile aggiornati.

B) File generato da `./config EXAM`:

Il file generato automaticamente dallo script di configurazione è `opt-exam.h`. Questo file **viene generato sempre**, in ogni singola compilazione, indipendentemente dalla presenza o meno dell'istruzione `options exam` all'interno del file di configurazione `EXAM`.

C) Contenuto del file generato:

Il file `opt-exam.h` contiene esclusivamente la definizione di una macro di preprocessore C che riflette lo stato dell'opzione:

- Se nel file EXAM è *presente* la voce `options exam`, il file conterrà: `#define OPT_EXAM 1`
- Se nel file EXAM *non è presente* tale voce, il file conterrà: `#define OPT_EXAM 0`

D) Correzione dell'inclusione condizionale in `main.c`:

L'utilizzo di `#ifdef OPT_EXAM` è **logicamente errato** nel contesto di OS161. Poiché il file `opt-exam.h` viene generato in ogni caso, la macro `OPT_EXAM` risulta sempre definita (con valore 0 o 1). Di conseguenza, la direttiva `#ifdef` valterebbe la condizione sempre come vera, compilando la chiamata a `exam_init()`; anche quando l'opzione è disabilitata.

La sintassi corretta deve valutare il *valore* logico della macro:

```
#if OPT_EXAM
    exam_init();
#endif
```

Inclusioni necessarie:

Per far sì che il compilatore possa espandere correttamente la macro `OPT_EXAM` e risolvere la firma della funzione, è strettamente necessario includere in testa al file C (es. `main.c`) le seguenti direttive:

1. `#include "opt-exam.h"` (per importare la macro `OPT_EXAM` a 0 o 1).
2. L'header file che espone il prototipo della funzione (es. `#include "syscall/exam.h"` se creato, oppure includere il prototipo manualmente).

Domanda 5 (Processi, Trapframe e Memoria Mappata)

A) Differenza tra Switchframe e Trapframe e il loro utilizzo:

La differenza risiede nella quantità di contesto hardware salvato e nel tipo di transizione:

- **Trapframe:** Salva l'intero stato della CPU (tutti i registri general-purpose, l'Exception Program Counter, i registri di stato). Viene generato dall'hardware ogniqualvolta si verifica un'eccezione (un evento asincrono o imprevedibile) che forza il passaggio da *User Mode* a *Kernel Mode*.
- **Switchframe:** Salva **solamente** i registri *callee-save* (i registri che la convenzione di chiamata del C impone alla funzione chiamata di preservare). Viene generato via software durante un cambio di contesto volontario e sincrono (*Context Switch*) tra due thread che stanno già girando all'interno del kernel.

Associazioni richieste:

1. **Avviare un processo utente: Trapframe.** Si costruisce un trapframe artificiale sullo stack del kernel e si esegue un ritorno da eccezione (`mips_usermode`) per forzare la CPU a declassare i privilegi ed entrare in User Mode.
2. **Gestire una chiamata di sistema: Trapframe.** La system call innesca una trap software (eccezione), richiedendo il salvataggio totale del contesto utente prima di eseguire il codice kernel.
3. **Gestire la thread_fork: Switchframe.** Crea l'impalcatura minima per il nuovo kernel thread, in modo che il dispatcher possa eseguirvi un *context switch* la prima volta che il thread ottiene la CPU.

B) Mappatura dei dispositivi di I/O in `kseg1` vs `kseg0`:

L'architettura MIPS suddivide la memoria in segmenti fisicamente differenti. I dispositivi hardware (Memory-Mapped I/O) sono mappati obbligatoriamente nel segmento `kseg1` perché esso fornisce accesso diretto e **non cachato** (uncached) alla memoria fisica.

Non sarebbe possibile mapparli in `kseg0` perché quest'ultimo è un segmento soggetto all'intervento della memoria Cache. Se i registri di un dispositivo I/O passassero per la cache:

- Le scritture (comandi verso il device) verrebbero trattenute nella cache della CPU e non raggiungerebbero l'hardware in tempo reale.
- Le letture (stato del device) preleverebbero dati obsoleti memorizzati in cache, perdendo i cambiamenti fisici generati dalla periferica.

C) Validità del buffer utente in `kseg0`:

NO, non è possibile.

Il segmento `kseg0` è un'area di memoria privilegiata riservata esclusivamente al kernel. Un processo utente (che opera nel segmento `kuseg`) non possiede i privilegi hardware per accedervi. Quando il processo invoca la system call `write(fd, buf, nb)` passando un puntatore appartenente a `kseg0`, il kernel **non deve fidarsi** dell'indirizzo.

La routine del kernel preposta a prelevare i dati (es. `copyin` in OS161) verificherà che l'indirizzo logico appartenga allo User Space. Rilevando che punta a `kseg0`, il kernel rifiuterà il trasferimento e farà fallire la system call restituendo un codice di errore di violazione di memoria (tipicamente `EFAULT`).